SpacemiT K3: A RVA23 RISC-V AI CPU with 60 TOPS AI Compute

SpacemiT Preview Version.

Abstract

This paper presents SpacemiT's second-generation edge CPU compute subsystem, K3, which adopts a homogeneous fusion paradigm: a high-performance RISC-V CPU cluster (X100) and an AI-first RISC-V AI-CPU cluster (A100) are integrated under a unified RISC-V ISA and run a single Linux operating system. K3 explores how a compact, co-designed hardware–software stack can deliver both general-purpose and AI capability within the constraints of resource-limited edge hardware. The paper describes the engineering choices and implementation of the X100 and A100, with particular emphasis on RVA23 enablement and hardware–software co-design for efficient LLM inference. It also outlines the OS scheduling and resource governance and AI software stack built on top of K3, forming an integrated solution for edge AI and robotics.

Keywords

RISC-V, AI-CPU, RVA23, Homogeneous Fusion, High-performance CPU, IME Extension, Trition

1 Introduction

As edge computing moves into a new phase driven by local LLMs and robotics, the key challenge is to deliver both strong general-purpose performance and strong AI performance in a single system, and to turn that into a scalable, cost-effective commercial product. To address this, we propose and implement a homogeneous fusion computing paradigm for the edge-AI era: under a unified RISC-V ISA and extension framework, we integrate a high-performance RISC-V CPU with an AI-optimized AI-CPU (or an ISA-integrated AI compute unit). Both run the same operating system (Linux) and support unified scheduling and resource governance with threads or tasks as the basic scheduling units, thereby enabling a system-level delivery model of "one platform, one stack, and one operations surface."

This work corresponds to our second-generation silicon. In the first generation, we validated the feasibility and customer value of homogeneous fusion in real deployments: on the one hand, the general-purpose RISC-V CPU ensures system and ecosystem compatibility, control-plane determinism, and engineering maintainability; on the other hand, the ISA-integrated AI compute resources can be invoked at low overhead within the same address space and process model, substantially reducing system complexity and ecosystem fragmentation for edge deployment. The evolutionary path is analogous to the mobile Internet era: to achieve both high performance and low power on smartphones, the industry introduced big.LITTLE designs on top of the Arm ecosystem while running a single operating system. In the AI era, our homogeneous fusion targets both high performance and high AI throughput, while compressing the system fragmentation cost introduced by heterogeneity into a controllable range.

SpacemiT, 2026/01/29
2026.

Compared with the first generation, the second-generation product improves along three clear and measurable dimensions:

- Stronger general-purpose compute: the second generation substantially improves CPU performance and system responsiveness, providing stronger single-thread and multi-thread capability to support edge control planes, system services, and complex application logic. Meanwhile, by anchoring ISA compatibility and software ecosystem alignment on RVA23, it achieves better toolchain and OS compatibility as well as long-term evolvability, providing a more robust platform foundation for continuous iteration of frameworks, drivers, and applications.
- Deployment scale: to meet model-side scaling requirements, it enables stable deployment and continuous evolution of larger LLMs or a greater number of agents on the edge.
- More efficient hardware–software co-design: including, but not limited to, a hierarchical memory system spanning on-chip SRAM and off-chip main memory (LPDDR) tailored for different application scenarios; low-latency execution paths for interactive workloads; and an end-to-end performance-driven heterogeneous scheduling strategy, avoiding characterizing real user experience solely by isolated kernels or peak TOPS.

The remainder of this paper is organized as follows: Section 3 describes the overall system architecture; Section 4 details the hardware design and key mechanisms; Section 5 introduces the software stack; and Section 6 reports the evaluation methodology and performance results.

2 System Overview and Design Rationale

2.1 Design Targets and System Trade-offs

To deliver, based on the RISC-V architecture, a mass-production AI CPU targeting two representative edge form factors—AI computers and AI robots—we ultimately converge the engineering constraints of the chip to the following three points:

- 12 nm process: adopt a 12 nm-class FinFET platform that has entered long-term volume production and is explicitly positioned for high-efficiency edge devices (e.g., 12FFC+ in volume production; N12e targeting AI-enabled IoT/edge devices), thereby reducing manufacturing and cost risk.
- Die area budget (<150 mm²): constrain die size to <150 mm² to reduce per-die cost and yield risk (widely used industry models indicate that die cost grows markedly super-linearly with area and yield decreases as die area increases), and to avoid dependence on supply-chain-constrained resources such as oversized packages and high-layer-count substrates.
- 30 W power: set TDP to 30 W, consistent with mainstream power envelopes of edge AI/robotics modules, enabling practical deployment within established thermal solutions and

product form factors; for example, the Jetson AGX Orin family provides power budget tiers such as 15 W / 30 W / 50 W.

Under the above constraints on process, area, and power, we further converge the architecture implementation to the compute-system design of the second-generation chip, with the following main components:

- 8 X100 cores: a high-performance RISC-V CPU subsystem fully compliant with RVA23, targeting control-plane determinism, ecosystem compatibility, and long-term maintainability for system services.
- 8 A100 cores: 60 TOPS-class general-purpose AI compute capability to serve as the primary engine for edge inference workloads.
- System-level enablers and scenario optimizations: close key system bottlenecks for end-to-end throughput, latency, and deployability, including:
  - Memory and high-speed I/O subsystem: e.g., 2×32-bit LPDDR5 controllers and a UCIe interface, maximizing effective bandwidth and expansion capability within edge area and power envelopes.
  - Low-latency response and control: e.g., a dual-RT24 design that partitions real-time tasks such as motor control and sensing; critical events are delivered and wake-up is triggered quickly via MSI interrupts, stabilizing latency and reducing jitter.
  - AI datapath-critical components: e.g., AI-DMA, reducing data-movement overhead, improving effective bandwidth utilization, and providing controllable primitives for resource governance under multi-agent and multi-model concurrency.

It is important to emphasize that designing a mass-production-grade edge AI CPU is fundamentally a software–hardware co-designed system endeavor. Satisfying hard hardware constraints—such as process node, area, and power—alone is insufficient to deliver a product at scale. Practical deployability and commercial usability are ultimately determined by the software stack, including the compiler/operator libraries and runtime, OS and drivers, and the toolchain for model deployment and performance analysis, as well as their co-optimized convergence with the underlying microarchitecture. Building on the software-stack assets and engineering validation accumulated from first-generation customer deployments, we treat software generality and usability as the primary gating constraint for the second-generation architecture design.

2.2 System Overview

This paper focuses on the K3 compute subsystem: a homogeneous-fusion compute platform composed of 2× clusters of X100 (general-purpose) and A100 (AI) cores. We describe its key capabilities including ISA/Profile configuration, the external and local interrupt subsystem (MSI/APIC), the instruction-trace infrastructure, and I/O isolation and virtualization via the IOMMU.

To deliver high general-purpose performance, the K3 SoC integrates 8 X100 cores across 2 clusters, providing an aggregate compute capability of 130K DMIPS. The X100 core is designed against the RISC-V application-processor standard RVA23 Profile, and fully implements all mandatory ISA extensions specified by RVA23U64 (user mode) and RVA23S64 (supervisor mode). In addition, the core proactively supports optional extensions including vector cryptography (e.g., Zvkng, Zvksg, Zvbc) as well as FP16/BF16 compute-related extensions such as Zfh, Zvfh, Zfbfmin, Zvfbfmin, Zvfbfwma. As a key standard within the RISC-V ecosystem, the RVA23 Profile aims to harmonize 64-bit application-processor implementations so that a binary software ecosystem can be built on a broadly guaranteed set of ISA extensions, while retaining flexibility through a small number of discoverable coarse-grained options. By being comprehensively RVA23-compatible, the X100 core ensures seamless compatibility with operating systems and applications built to this specification, enabling rapid integration into the increasingly mature RISC-V software ecosystem.

Table 1: K3 AI compute.

K3	Dense (TOPS)	Sparse (TOPS)	Notes
INT4	30	60	Supports block scaling to preserve the accuracy of INT4-quantized models
INT8	15	30	Accelerates convolution to improve conv-kernel throughput
BF16/ FP16	7.5	N/A	Supports both IEEE FP16 and bfloat16 (Brain Float 16) formats
FP8	7.5	N/A	FP8 operands are converted to BF16 before computatio

To deliver high AI performance, the K3 SoC integrates 8 A100 cores (2 clusters), providing 60 TOPS of AI compute. The A100 is likewise built on the standardized ISA/toolchain baseline defined by the RVA23 Profile: on the one hand, RVA23U64 elevates V (Vector) to a mandatory extension, together with mandatory vector sub-extensions such as Zvfhmin/Zvbb/Zvkt; on the other hand, RVA23U64 lists BF16-related extensions that are closely tied to AI mixed-precision—Zfbfmin, Zvfbfmin, Zvfbfwma—as optional, providing a discoverable and portable standard interface path for the evolution of AI/ML data types and operators. The typical AI compute specifications provided by A100 on K3 are summarized in the Table 1.

To support MSI (Message-Signaled Interrupt)-based external interrupts as well as local interrupts (software interrupts and timer interrupts), the K3 SoC integrates SpacemiT's in-house interrupt controller, APIC. APIC is composed of APLIC and ACLINT:

- APLIC (Advanced Platform-Level Interrupt Controller) follows The RISC-V Advanced Interrupt Architecture
- ACLINT (Advanced Core Local Interruptor) follows RISC-V Advanced Core Local Interruptor (ACLINT) Specification

MSI realizes "external interrupts" as a device-issued memory-write message to a designated address. Mechanistically, this avoids common issues of legacy pin-based interrupts such as shared IRQ lines and the associated overhead of scanning multiple handlers under IRQ sharing. MSI can also leverage the ordering semantics of message delivery, helping reduce extra synchronization or readback paths that may otherwise be needed when an interrupt is observed

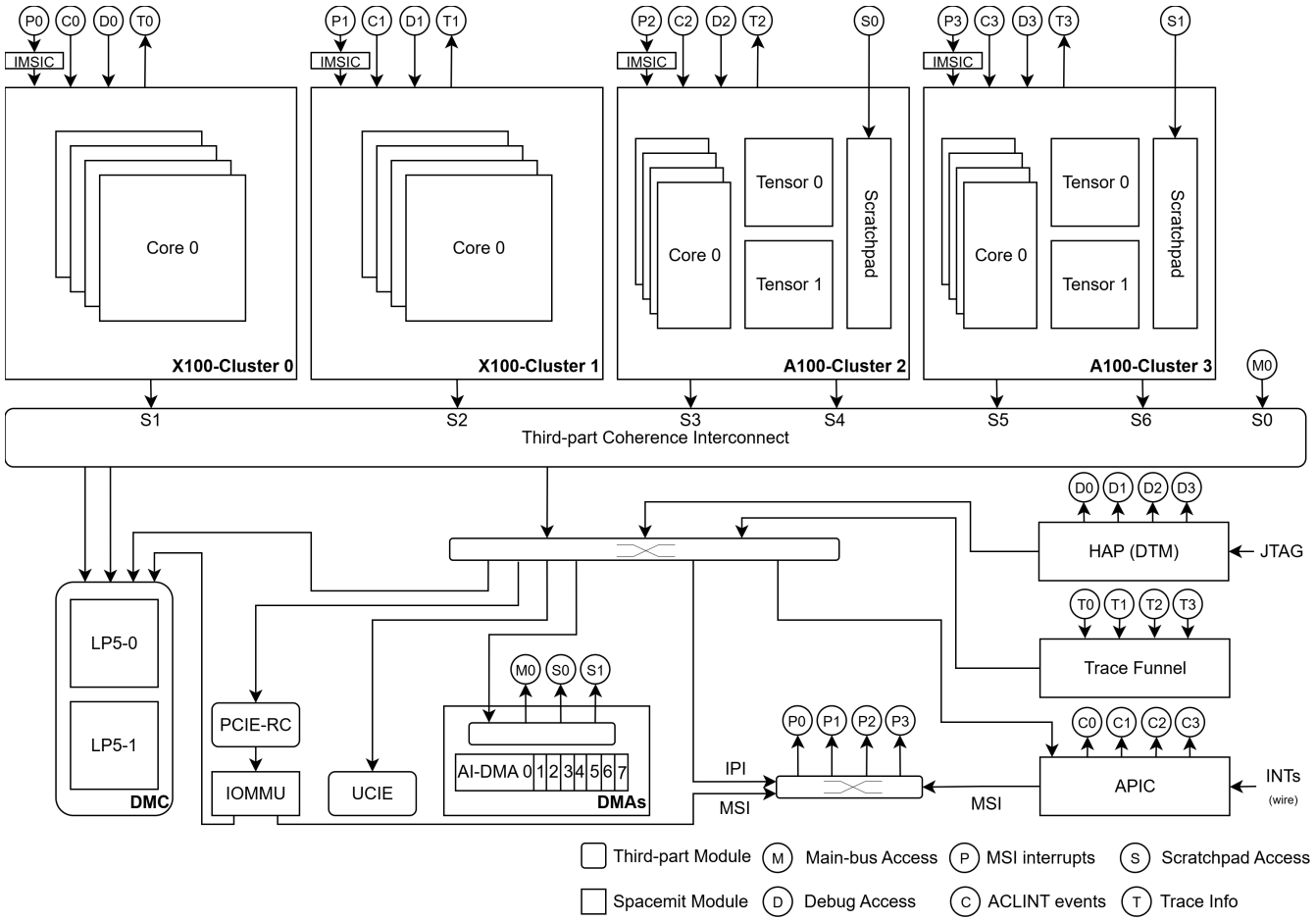

Figure 1: System Overview of K3.

before its corresponding data writes become visible. Furthermore, MSI/MSI-X supports multiple interrupt vectors and can be configured to target specific CPUs/cores, which is beneficial for robotics workloads: interrupts from critical sensors and control loops can be steered to isolated cores to reduce jitter and improve real-time determinism.

To enable instruction tracing (Trace) for processor cores, the K3 SoC integrates RISC-V Trace encoders and corresponding sink components for both X100 and A100. The instruction-trace interface of X100 and A100 complies with the RISC-V Hart-to-Trace Interface specification; the trace encoder is designed against the RISC-V N-Trace (Nexus-based Trace) Specification (Version 1.0); and the trace component register definitions are compatible with the RISC-V Trace Control Interface Specification (Version 1.0), supporting configuration via the system interconnect. The X100/A100 trace encoders implement the following standard features:

- BTM-mode encoded output;
- Trace enable/disable and event emission based on Debug Triggers;
- Address-compression–based encoding optimizations;
- Event transport on Trace data overflow;
- Ownership event messages, facilitating differentiation and tracking of process execution flows in Linux and other OSes.

For secure isolation and manageability of the I/O subsystem, the K3 SoC integrates SpacemiT's in-house T100 IOMMU. T100 follows the RISC-V IOMMU Architecture Specification and the RISC-V Server SoC Specification. At the hardware level, it provides device-DMA address translation, access control, and isolation protection; for virtualization, it further offers hardware-assisted I/O virtualization, supporting device passthrough (PCIe passthrough) and device virtualization (e.g., SR-IOV).

3 Hardware Design

In the K3 computing system, X100 and A100 achieve interoperability under a unified RISC-V architectural framework; the immediate value is manifested in the ability for general-purpose and AI workloads to execute cooperatively within the same software stack. More importantly, this approach avoids the coexistence of a "dual-OS / dual-stack" configuration typical of conventional heterogeneous architectures, along with the resulting resource footprint, boundary-crossing overhead, and maintenance complexity.

We observe that the AI software stack is continuously evolving toward greater complexity and deeper reliance on open-source dependencies: spanning kernels and device drivers, containers and runtimes, compilers and frameworks, and the broader model ecosystem, key capabilities are increasingly composed from a set of rapidly evolving upstream open-source components. This trend makes it difficult for a hardware vendor to sustain, over the long term, an approach that "independently rebuilds a closed, proprietary stack"; instead, system software must be able to track upstream community iterations and upgrades over time in order to remain competitive in functionality, performance, and security. In this context, adopting Linux as the underlying operating system to host and integrate these open-source components is the most natural and most sustainable choice for constructing an AI software stack.

Within today's mainstream RISC-V software ecosystem, the host CPU domain typically uses Linux as the general-purpose software substrate; meanwhile, in order to support AI frameworks and runtimes and to maintain long-term compatibility with upstream open-source components, the device-side/accelerator-side AI software stack is, from an execution unit perspective, also effectively compelled to adopt Linux. If one follows the traditional heterogeneous "host CPU + accelerator" design, this is essentially equivalent to running two largely identical Linux instances within the same system (with substantial overlap in both the kernel and baseline user-space components) and coordinating them via cross-boundary mechanisms. Such duplicated stacks not only incur additional resource consumption (memory, storage, and resident services), but also introduce boundary-induced data-path and synchronization overheads (IPC, data copies, and context switches), and significantly increase the complexity of version maintenance, security patching, and the verification matrix—thereby undermining overall system efficiency, maintainability, and long-term evolvability.

3.1 X100

X100 is a high-performance processor core based on the RISC-V instruction set architecture, providing full compliance with the RVA23 profile. It adopts a 4-wide dispatch, out-of-order microarchitecture, achieving high compute performance while balancing energy efficiency and die-area cost. This design effectively targets high-performance edge workloads and is also suitable for deployment in entry-level servers. To meet the requirements of high-performance computing and scalable multicore integration, X100 incorporates key features including multicore interconnect, virtualization support, reliability/ availability/ serviceability (RAS), and security mechanisms.

From an architectural standpoint, X100 takes OpenC910 (the open-source release of Xuantie C910) as the initial reference point, and introduces extensive optimizations and functional enhancements across multiple subsystems. The major improvements include: increasing frontend fetch bandwidth and improving branch prediction accuracy; widening decode and dispatch and leveraging multiple techniques to enhance execution efficiency; restructuring the load/store subsystem to increase memory bandwidth while substantially reducing access latency; adding a vector execution unit compliant with the RISC-V Vector Extension v1.0; and refining the multicore interconnect to improve scalability and coherence support.

Beyond performance, X100 maintains a high degree of conformance and forward-compatibility in ISA support. For general-purpose computing, X100 does not introduce any additional proprietary ISA extensions, relying exclusively on the official RISC-V standard instruction set. Building on this foundation, it tracks the latest industry-standard evolution by supporting all mandatory components of the RVA23 profile, while also implementing selected optional extensions and additional extensions not explicitly enumerated in the profile, ensuring adaptability to future software-ecosystem and hardware-platform evolution.

3.1.1 Frontend.

The increase in dispatch width of the X100 imposes greater demand on instruction fetch bandwidth at the front-end. To address this, the X100 processor core implements a series of key microarchitectural optimizations aimed at significantly enhancing the effective instruction fetch bandwidth. These enhancements ensure a sustained and efficient supply of the instruction stream to the backend execution units. The optimizations are primarily focused on two critical dimensions: improving branch predictor performance and increasing fetch parallelism.

Branch Target Prediction: The X100 incorporates targeted optimizations to the timing path of the L0 Branch Target Buffer (BTB) and moderately expands its entry capacity. Furthermore, by introducing a more sophisticated replacement algorithm better suited to the locality characteristics of program branch accesses, the hit rate of the L0 BTB is effectively improved. This enhancement directly reduces front-end bubbles and the associated fetch pipeline disruptions caused by target address mispredictions.

Conditional Branch Direction Prediction: For conditional branch direction prediction, the X100 employs a more advanced TAGE (Tagged Geometric History Length) predictor, superseding the previous Bi-Mode algorithm. The TAGE predictor leverages long global history patterns to make predictions, significantly improving the prediction accuracy for complex control-flow behaviors.

Return Address Prediction: For function call returns, the X100 implements a Return Address Stack (RAS) based on a linked-list structure. This design more robustly supports return address prediction in out-of-order execution scenarios involving multithreading or deeply nested function calls, thereby improving the prediction performance for return instructions.

Instruction Fetch Bandwidth Optimization: To improve fetch bandwidth, the X100 expands the organization of the Instruction Cache (ICache) from 2-way to 4-way set-associative. This modification effectively reduces the probability of cache misses due to way conflicts, thereby enhancing the continuity of instruction supply. To further mitigate access latency, the X100 increases the data transfer bandwidth between the ICache and the L2 Cache from 128 bits to 256 bits per cycle. This reduces contention and occupancy of the access channel to the lower-level cache.

Instruction Dispatch and Fusion: To fully leverage the backend's out-of-order execution capabilities, the X100 widens the instruction dispatch bandwidth from three to four instructions per cycle. This ensures that the decode and dispatch stages can continuously supply a sufficient stream of instructions to the backend. Additionally,

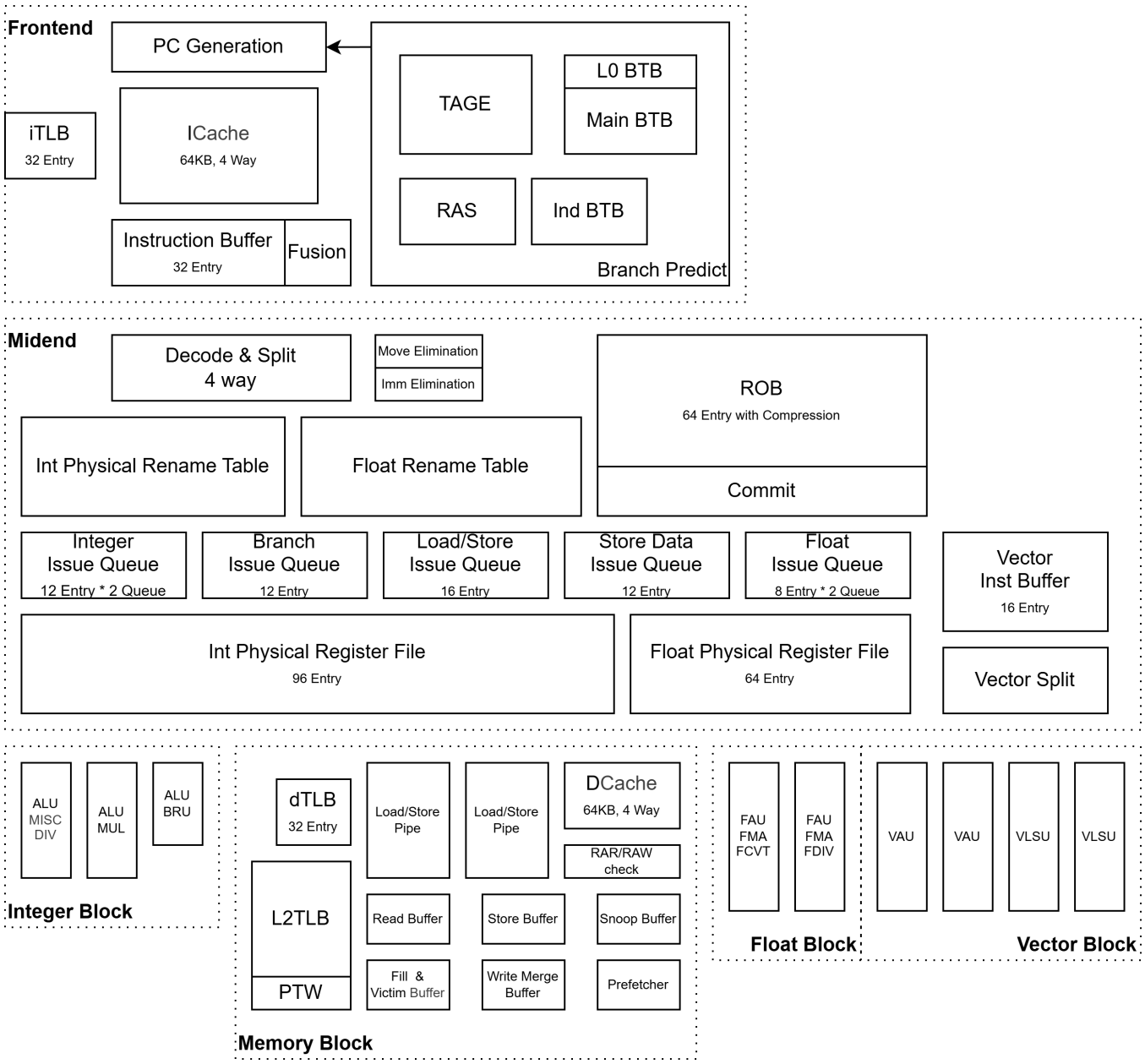

Figure 2: Block diagram of X100 core.

the X100 front-end integrates multiple statically determined instruction fusion patterns. These patterns combine common instruction sequences into more efficient micro-operations. This instruction fusion mechanism effectively increases the useful instruction dispatch bandwidth without introducing new ISA extensions, thereby maintaining full RISC-V compatibility.

Reliability Enhancements: Beyond performance improvements, the X100 front-end also incorporates reliability enhancements. Given that dirty cache lines cannot exist in an ICache, the X100 introduces a parity check mechanism for the ICache. This mechanism enables real-time detection of single-bit errors in cached instruction data, significantly strengthening the reliability of instruction storage. Concurrently, when an error is detected at the front-end, the error information is reported via the official RISC-V RERI (Reliability, Availability, and Serviceability Error Reporting Interface). This capability allows for the accurate logging and reporting of various front-end reliability events, providing crucial support for system-level diagnostics and recovery mechanisms. Collectively, these features reinforce the overall RAS (Reliability, Availability, and Serviceability) characteristics of the X100 processor.

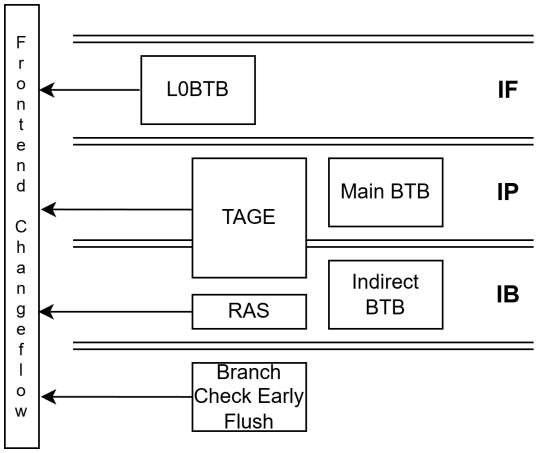

Figure 3: Pipeline of X100 frontend branch predictor.

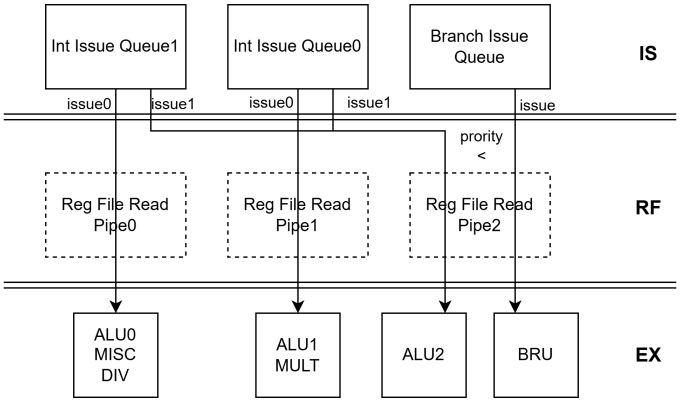

Figure 4: X100 3-ALU pipeline structure.

3.1.2 *Midcore.*

The mid-core encompasses the stages of instruction decode, rename, dispatch, and issue. A primary focus of our microarchitectural redesign at this stage was the transition from a 3-wide to an optimized 4-wide pipeline structure.

In the baseline OpenC910 architecture, the Instruction Decode (ID) stage was 3-wide, while the subsequent Instruction Ready (IR) stage had already been expanded to 4-wide to accommodate potential instruction splits originating from the ID stage. To align front-end throughput with backend capacity, we redesigned the decode logic to support 4-way parallel decoding.

The increase in decode width necessitated corresponding enhancements in downstream pipeline structures to handle the higher instruction throughput. We accordingly increased the depth of the Issue Queue to accommodate a larger number of in-flight instructions, thereby preventing structural hazards caused by queue capacity limitations. Concurrently, we integrated an additional Arithmetic Logic Unit (ALU) to mitigate structural conflicts arising from insufficient execution resources.

The widening of the pipeline, deepening of buffers, and addition of execution units invariably introduce deterministic timing challenges. To alleviate potential critical path degradation associated with these expansions, we adopted a resource-sharing strategy for the new ALU. Instead of adding dedicated register file read ports, the new ALU reuses the read ports of the Branch Processing Unit (BRU). Specifically, when no branch instruction is being issued, the two integer issue queues can utilize the BRU's register read ports to dispatch instructions to the additional ALU. This port-sharing approach enhances parallel execution capability while effectively containing the growth of critical timing paths, mitigating the risk of timing deterioration.

To further exploit the microarchitecture's out-of-order execution potential, we implemented several key optimizations within the mid-core: 1. Immediate Operand Elimination: Logic was added to eliminate the allocation of physical registers for certain immediate operands, thereby alleviating register file pressure. 2. Early Wake-Up: A strategy was adopted to release dependent resources early during the execution of long-latency operations (e.g., division or L1 cache misses), effectively reducing instruction stall time. 3. Early Front-End Resource Release on Mispredict: Resources in the front-end pipeline are released earlier on mispredicted paths, reducing the performance penalty associated with branch mispredictions. 4. Enhanced Issue Queue Management: The balancing policy of the Issue Queue was modified, and an early deallocation mechanism was introduced. These changes improve the utilization efficiency of the Issue Queue, effectively providing additional queue depth. Collectively, these optimizations significantly enhance the processor's instruction scheduling efficiency and resource management capabilities during out-of-order execution.

3.1.3 *Memory Subsystem.*

The memory subsystem constitutes a critical component of processor microarchitecture, responsible for address generation of memory instructions, virtual-to-physical address translation, cache lookups, and scheduling of off-chip memory access transactions. Its performance, typically measured by bandwidth and latency metrics, represents one of the primary bottlenecks limiting overall processor efficiency in modern designs. The X100 processor features a redesigned memory subsystem that demonstrates significant improvements over its predecessor architecture in execution bandwidth, bus bandwidth, and bus latency.

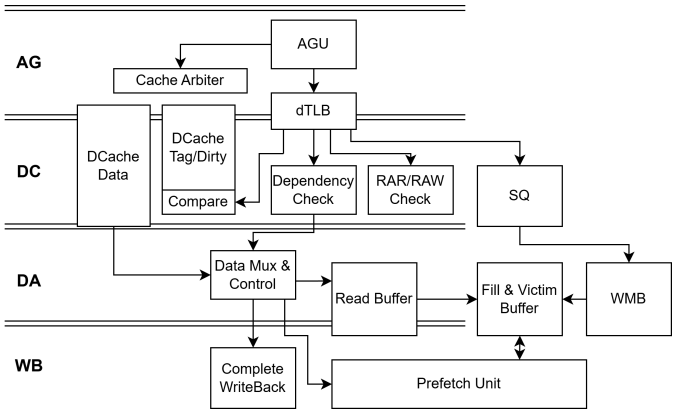

Figure 5: Mirco architecture of X100 Memory Subsystem.

The memory subsystem constitutes a critical component of processor microarchitecture, responsible for address generation of memory instructions, virtual-to-physical address translation, cache lookups, and scheduling of off-chip memory access transactions. Its performance, typically measured by bandwidth and latency metrics, represents one of the primary bottlenecks limiting overall processor efficiency in modern designs. The X100 processor features a redesigned memory subsystem that demonstrates significant improvements over its predecessor architecture in execution bandwidth, bus bandwidth, and bus latency.

The X100 incorporates two general-purpose pipelines, each equipped with dedicated load/store units. Correspondingly, the data cache (DCache) structure and organization were reconfigured to double its access bandwidth. However, increased bandwidth often elevates the out-of-order execution of memory instructions, potentially introducing more memory hazards—particularly false Read-After-Read (RAR) and Read-After-Write (RAW) hazards—that incur substantial pipeline flush and refill penalties. To mitigate this issue, the X100 implements a snoop monitor to filter false hazards and employs a micro-operation (uop)-granularity hazard predictor. This predictor proactively identifies potential data dependency conflicts and delays the execution of conflicting instructions, thereby significantly reducing latency penalties associated with memory hazards.

The X100 upgrades the bus width to 256 bits and utilizes a CHI-like protocol to interface with the multi-core interconnect unit, enhancing interconnection efficiency. Key optimizations include restructuring several critical buffers:

- The Write Merge Buffer (WMB) was redesigned to improve merging efficiency and bus utilization while maintaining the same entry count.
- The Line Fill Buffer (LFB) and Victim Buffer (VB) were consolidated into a unified Fill & Victim Buffer (FVB). This resource-sharing approach reduces total buffer resource overhead while improving utilization efficiency, addressing structural conflicts that previously limited outstanding request capacity and thereby enhancing the core's overall outstanding transaction capability.

These buffer optimizations enable concurrent multi-line accesses under the same cache index in both WMB and FVB, substantially improving bus bandwidth utilization in data-intensive workloads such as AI tensor operations.

The X100 implements several latency-focused optimizations:

- An early wake-up mechanism reduces load-to-use latency in L1 miss/L2 hit scenarios.
- Atomic memory operations (AMOs) are executed more atomically without instruction splitting, significantly lowering AMO execution latency and reducing live-lock probability during lock acquisition.
- The data prefetcher incorporates a Stream-based Page Prefetching (SPP) algorithm to improve coverage of irregular access patterns, combined with a bus load feedback mechanism that dynamically adjusts prefetch distance to balance timeliness against bandwidth consumption.

The X100 adds support for the Hypervisor virtualization extension. To improve multi-level page table walk efficiency, it caches intermediate page table entries in the TLB, effectively reducing address translation overhead in virtualized environments. The design also implements ECC protection for the DCache and supports handling Poison reporting from Data Check errors when supported by the external bus interface, alongside compliance with the RISC-V standard Reliability, Availability, and Serviceability Error Reporting Interface (RERI). These features provide essential support for server-class applications.

3.1.4 Vector.

X100 integrates an efficient vector processing subsystem that delivers significant performance benefits for data-parallel workloads. This subsystem is characterized by its full compliance with the RISC-V Vector Extension Version 1.0 (RVV 1.0) and the RISC-V Vector Crypto Extension to address the demands of data-intensive computing and high-performance security applications.

Fundamentally, the X100 vector unit adheres to RVV1.0 with a fixed vector length (VLEN) of 256 bits and maximum element length (ELEN) of 64 bits, supporting data types from INT8 to INT64 and FP16/BF16 to FP64. The architecture further enhances throughput by featuring independent vector pipelines , including two arithmetic pipelines and two load/store pipelines with each pipe dealing with 128-bit data, which can operate concurrently to maximize data processing efficiency.

Decoupled Microarchitecture. At the microarchitectural level, the X100 VPU employs a decoupled approach relative to the scalar core. After initial decoding and scalar operand renaming within the main pipeline, vector instructions are dispatched to an independent vector unit for subsequent splitting and execution. Compared to the tightly-interconnected modules of the scalar core, the vector unit is functionally more separated and occupies a significant silicon area. A decoupled vector unit not only provides enhanced flexibility and configurability but also simplifies physical layout and floorplanning for the core.

At the architectural level, the X100 vector employs a decoupled design strategy. After undergoing initial decoding and scalar operand renaming in the main pipeline, vector instructions are dispatched to a dedicated, independent vector unit for subsequent issue,split and execution. This relative decoupled design is motivated by the observation that the vector unit, compared to the tightly-interconnected modules of the scalar core, is functionally more separated and occupies a significant silicon area. A decoupled vector unit not only provides enhanced flexibility and configurability but also simplifies physical layout and floorplanning for the core.

Vector instructions are first pushed in a dedicated vector instruction buffer, which serves to decouple them from the main pipeline. Instructions are then issued from this buffer for further processing. The vector unit supports the concurrent issue of up to two memory instructions and two computational instructions. Each issue port can handle an instruction micro-op in 128-bit granularity per cycle, which means an instruction with LMUL=1 is issued as two 128-bit micro-ops. The design choice of employing dual 128-bit data paths, as opposed to a single 256-bit path, is primarily driven by the need to balance hardware resource costs effectively. Given the wide variety and high computational complexity of vector operations, a finer-grained datapath width enables more efficient and balanced

allocation of resources among different instruction types. For instance, complex permutation operations may have only one 128-bit execution unit and when two such instructions are issued concurrently, only the older one can acquire the resource. This granularity also facilitates parallelism between distinct functional resources, allowing, for example, an addition instruction and a shift instruction to be issued simultaneously and executed on their respective units.

Memory access efficiency is paramount for vector applications. The X100 decouples address generation and data handling for vector load/store instructions: the address phase is processed by the main pipeline's Load-Store Unit (LSU) for address calculation and memory access initiation, while the data phase is handled by the vector unit for operand splitting and vector register dependency checking. Communication and data exchange between the two are coordinated via instruction IDs. The LSU leverages its two existing memory pipelines to issue and process two vector memory instructions in parallel. Correspondingly, the vector unit is equipped with two matching splitter circuits to handle the data paths of these two memory instructions. By decoupling address and data processing, the main pipeline can forward instructions to the LSU as soon as addresses are ready, without waiting for vector register dependencies to be resolved. This allows the LSU to initiate memory accesses promptly, thereby accelerating data retrieval and improving overall memory efficiency. Furthermore, for handling complex vector memory patterns such as segment accesses, the LSU incorporates a dedicated transpose buffer to accelerate data reorganization, significantly boosting execution efficiency.

3.1.5 Multi-core Interconnect.

The X100 architecture implements a systematic and comprehensive upgrade to its multi-core interconnect system, targeting key bottlenecks in high-performance computing and server workloads. This upgrade achieves co-optimization across latency, bandwidth, and overall system efficiency, spanning critical layers including the cache hierarchy, memory subsystem, reliability mechanisms, and coherence protocol.

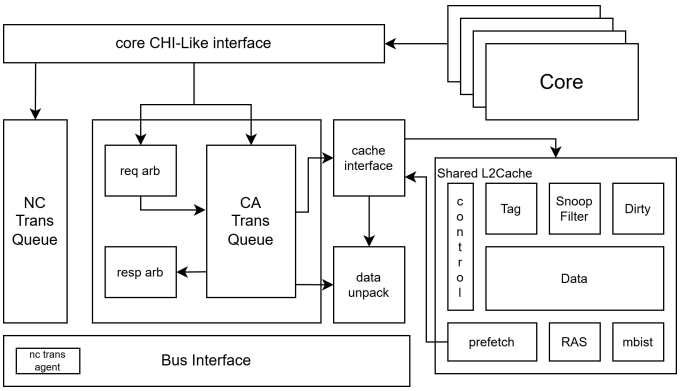

Figure 6: Multi-core Interconnect of X100.

The X100 transitions from an inclusive to an exclusive cache policy. Furthermore, it introduces a superior cache tag backup mechanism for each core, enabling precise tracking of cache line states across the multi-core environment. By eliminating redundant data copies between cache levels, this approach effectively increases usable cache capacity. A 1:1.5 expansion ratio is employed to reduce the frequency of back-invalidations. In terms of latency optimization, the architecture enables early access to the L2 cache, with pipeline operations canceled upon detected address conflicts. This optimization significantly reduces the critical load-to-use latency, as return data can be delivered directly to the core, bypassing the additional latency introduced by access transaction queues. Following these optimizations, the load-to-use latency for an L1 miss with an L2 hit can be as low as 17 cycles, accelerating the resolution of data dependencies. The prefetching mechanism has also been enhanced. The memory subsystem integrates a next-line prefetcher and a best-offset prefetcher. The former is trained on instruction and page table walk (PTW) misses, while the latter is trained on data access patterns. This combined approach improves both spatial locality prediction and adaptability to complex access strides, thereby boosting the overall cache hit rate.

The memory subsystem doubles the number of cache banks, supporting parallel access to up to 4 banks. This significantly improves memory-level parallelism and internal data bandwidth. To match this design, the internal bus width has been expanded to 256 bits, further unleashing the potential of memory bandwidth.

The multi-core coherence protocol benefits from several key optimizations. Bypass optimization techniques reduce the latency of common snoop transactions by 4 cycles. The architecture supports self-snooping, effectively resolving cache aliasing issues within a core, and enables precise bank snooping to reduce power consumption. The internal state machine of the transaction queue has been redesigned, shortening the lifecycle of common multi-core transactions and improving overall interconnect efficiency. For high-concurrency lock acquisition scenarios, an exclusive CleanUnique transaction type has been introduced. This strategically avoids live-lock issues under heavy load, enhancing system stability in highly competitive environments.

For server-grade Reliability, Availability, and Serviceability (RAS), the X100 introduces a standard error reporting mechanism compliant with the RISC-V RERI specification. This is integrated with the foundational software layer to enable more accurate error handling and system recovery. Regarding data integrity, the architecture implements a poison mechanism for error tagging and propagation. This is combined with data check technology to detect bus transmission errors. The system can flexibly choose between delayed reporting of error information via the RERI interface or immediate return of traditional error signals. This flexibility allows the system to balance reliability requirements with real-time performance constraints.

3.2 A100

A100 is an AI-centric RISC-V AI-CPU that delivers native AI compute capability through the SpacemiT-IME instruction-set extensions. Its microarchitecture is deliberately optimized for operator-level parallelism, memory bandwidth, and data locality to ensure efficient execution of real-world AI workloads. In addition, it fully supports the general-purpose CPU functionality defined by RVA23 (excluding the Hypervisor extension), thereby enabling large-model inference and a broad range of AI applications under a standard

RISC-V unified programming model. The hardware design is developed along three primary axes:

- System Dimension (Integration): The AI unit is embedded into the CPU pipeline as a tightly coupled coprocessor under a single-ISA model, reducing driver overhead and context-switch latency, and eliminating the "heterogeneity tax."
- Computation Dimension (Computation): Targeting Transformer workloads, the design pursues model–hardware co-design; via a vectorized microarchitecture and enhanced low-/mixed-precision arithmetic, it increases operator parallelism and improves energy efficiency.
- Memory/Datapath Dimension (Dataflow): The dataflow and microarchitecture are planned with bandwidth as the central objective, and the design implements software-managed local memory and decoupled access–execute operation—specifically, overlapping memory movement with computation through asynchronous DMA prefetch, thereby reducing execution stalls.

Each cluster comprises four scalar cores, two vector cores, two local-memory banks, and a shared L2 cache. The two vector cores share a single Tensor Core and one local-memory bank, with the goal of improving tensor-compute resource utilization and data-reuse efficiency under stringent area and bandwidth budgets. The K3 chip integrates two A100 clusters.

3.2.1 *Vector.*

The A100 Vector Unit is driven by two parallel vector pipelines that perform vector-instruction splitting, dependency handling, and vector register-file reads/writes, with results produced primarily in stages VE3–VE5. From an execution-organization perspective, the Vector Unit classifies operators into five categories according to "instruction types + datapath characteristics": INT, FP, CRYPTO, FDV, and IME-1.0.

The register file is provisioned with VLEN = 1024-bit vector registers, comprising 32 architectural vector registers (v0–v31). This register width determines that a single vector instruction can operate on up to 1024 bits of data per instruction instance, substantially increasing achievable throughput.

Compute-resource provisioning follows a simple guiding policy. High-frequency, throughput-critical operators (add/sub, compare, shift, logic, and narrow-width multiply–accumulate) are preferentially mapped to wider execution resources capable of dual-issue, and are scheduled to produce results as early as possible in VE3 (or VE4). Mixed execution widths follow the principle of "fixing width by mainstream precisions, slicing/packing by precision, and reusing resources for rare widths": mainstream data types such as INT8/INT16/BF16 define the critical datapath widths and lane counts; lane slicing and element packing increase parallelism for narrow precisions; and the relatively infrequent wider datapaths are realized via cross-lane merging or multi-cycle time-multiplexing to avoid over-provisioning for peak width. Low-frequency or structurally complex operators (e.g., permute/reorder, partial wide-width datapaths, and iterative division) deliberately constrain issue bandwidth and datapath width to improve area and energy efficiency. Table 2 summarizes the A100 compute-resource configuration using a unified notation of "number of channels × per-channel bit-width":

SpacemiT-IME provides a two-tier matrix multiply–accumulate capability. The per-core private IME-1.0 execution unit reuses the Vector Unit's low-throughput datapath with VLEN = 256/512, targeting lightweight small-matrix MAC operations of INT8×INT8→INT32. In contrast, the dual-core shared IME-2.0 execution unit adopts a dedicated high-throughput datapath with VLEN = 1024/2048, targeting higher throughput requirements and covering matrix MACs across multiple mixed data types and mixed-precision modes. Table 3 summarizes the supported data types and matrix-shape definitions in a unified manner.

Implementing RVV VLEN = 1024 (an ultra-wide datapath) on a 12 nm process makes timing closure often difficult to achieve in a single pass with "default synthesis" alone and typically requires targeted iteration in conjunction with physical implementation. For example, the register-file read/write ports and the bypass/selection network may be structurally partitioned and/or hierarchically pipelined; critical paths are then compressed via retiming and fanout control. In the backend, floorplanning, critical-block placement, and constraint-driven guidance are used to reduce timing degradation caused by long interconnects and congestion. Ultimately, A100 closes PPA on the VLEN=1024–related critical paths at 2.1 GHz.

3.2.2 *Tensor Core.*

The Tensor Core is shared by two compute cores and can accept one matrix multiply–accumulate (MMA) instruction per cycle from each core. Oprands and results of MMA instructions are stored in vector registers. The vector subsystem provides 32 vector registers, each with a width of 128B. Each core is equipped with an independent 512-bit load channel. The front end supports dual issue in the form of one 512-bit load + one MMA instruction per cycle. We define the operational intensity (a.k.a. compute-to-memory ratio) as $I = \frac{W}{Q}$ (MAC/Byte), where Q denotes the number of bytes transferred from memory into vector registers. To approximate the steady-state utilization of the MAC unit, we use $U = \frac{N_{\text{mac}}}{\max(N_{\text{mac}}, N_{\text{ld}})}$. where N_{mac} and N_{ld} denote, respectively, the number of cycles spent issuing MMA instructions and memory loads. Consider an INT8 to INT32 MMA microkernel with parameters M = N = 8, and K=16. The total number of operations per MMA is $W = M\dot{N}\dot{K} = 1024\ MAC$. Case 1: Single MMA instruction (no tiling / no reuse): Matrix A contains 8×16 INT8 elements, i.e., 128 B, which requires two 512-bit load cycles. Each MMA consumes a fresh instance of A (i.e., no reuse across MMA invocations): $U = 1/2 = 50\%$. The memory traffic in this case:Q = 128B, which leads to $I = \frac{1024}{128} = 8$ MAC/B. Case 2: Tiling with panelized loads and register blocking: We next consider a tiled execution in which both A and B are panel-loaded and then reused via register-level $m_R \times n_R$ register blocking, thereby amplifying data reuse. Assume 16 vector registers are allocated to hold C-tiles. Each C-tile requires two vector registers, and 8 C-tiles are stored in vector registers simultaneously. Let the number of simultaneously accumulated tiles be $r_m \times r_n$ with the constraint $r_m r_n \leq 8$. Under this scheme, r_m panels of A and r_n panels of B must be loaded. $r_m r_n$ MMA operations are conducted and results are accumulated into the resident C-tiles. Consequently, $Q = 128(r_m + r_n)$, $W = 1024\ r_m r_n$, $I = 8 \cdot \frac{r_m r_n}{r_m + r_n}$. Under the constraint $r_m r_n \leq 8$, the maximum operational intensity is achieved at $(r_m, r_n) = (2, 4)$, yielding $I_{max} \approx 10.67$ MAC/B. For the maximizer, the load and MAC cycles are $N_{\text{ld}} = 12$ and $N_{\text{mac}} = 8$, and thus$U = \frac{8}{12} \approx 66.7\%$.

Figure 7: Block diagram of A100.

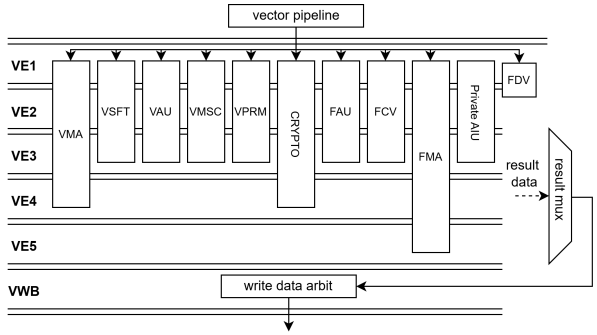

Figure 8: A100 Vector Execution Unit.

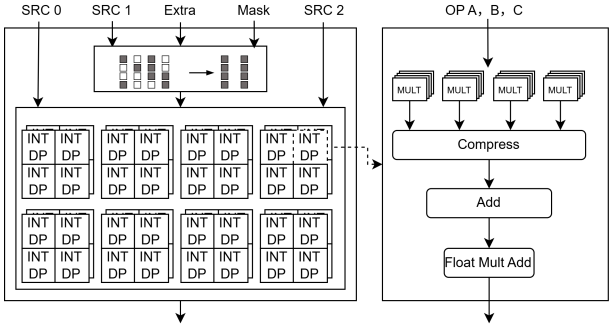

Figure 9: Tensor Core of A100.

Domain	Unit / Condition	Resource Provisioning (channels × width)	Result Stage
INT	VAU (add/sub/compare/minmax)	2×256	VE3
INT	VSFT (shift/rotate/sat-shift)	2×256	VE3
INT	VMA (SEW=8/16)	2×256	VE3
INT	VMA (SEW=32/64)	2×128	VE4
INT	VMSC (mov/logic)	2×256	VE3
INT	VMSC (others, e.g., clz/cpop)	2×128	VE3
INT	VPRM (slide/gather/compress/transpose, etc.)	2×256	Implementation-dependent
FP	FAU (SEW=16/32)	2×256	VE3
FP	FAU (SEW=64)	2×128	VE3
FP	FMA (SEW=16/32)	2×256	VE4
FP	FMA (SEW=64)	2×128	VE5
FP	FCV (SEW=16/32)	2×256	VE3
FP	FCV (SEW=64)	2×128	VE3
CRYPTO	128-bit group (AES/GHASH / subset of SHA-2)	2×256	VE3
CRYPTO	256-bit group (SM3/SM4 / subset of SHA-2)	2×128	VE4
FDV	div/rem/sqrt (SRT iteration; shared int/FP resources)	1×128	At iteration completion

Table 2: Execution resource of A100 Vector Execution Unit.

Unit	Mode	Execution Width	Operation	AB	CD
IME 1.0	Per-core private	256-bit	$D = A \cdot B + C$	$(A =)$ i8 (4×8), $(B =)$ i8 (8×4)	$(C/D =)$ i32 (4×4)
IME 2.0 (INT)	Shared by two cores	1024-bit	$D = A \cdot B + C$	i4 : $(A)(8 \times 32)$, $(B)(32 \times 8)$ i8 : $(A)(8 \times 16)$, $(B)(16 \times 8)$	–
IME 2.0 (FP)	Shared by two cores	1024-bit	$D = A \cdot B + C$	$(A/B =)$ fp16 or bf16 (8×8)	$(C/D =)$ fp32 (8×8)

Table 3: SpacemiT-IME execution modes and supported MMA shapes.

Scenario	Single Core: (U)	Single Core: (I) (MAC/B)	Dual-core shared MAC: total utilization (U_{total}) (normalized to 1 MAC/cycle)	Dual-core shared MAC: system-level(I) (MAC/B)
Single MAC instruction (A=1024-bit)	50%	8	100%	8
Tiling with register blocking (16 vprs for C-tile, $r_m = 2$, $r_n = 4$)	66.70%	10.67	100%	10.67

Table 4: MAC utilization and system-level efficiency under different execution scenarios.

Note: The effective throughput of the shared MAC engine saturates to 1 MAC/cycle when the aggregate MAC throughput by the two cores exceeds 1 MAC/cycle. For this $8 \times 16 \times 8$ microkernels, even if a single core achieves the operational intensity of ≈ 10.67 via tiling, the rate at which it can generate MAC requests may still fall short of the peak service rate of the shared MAC unit, thereby leaving the area- and power-expensive MAC resources underutilized. In contrast, when two cores can load data in parallel and continuously issue MMA instructions to the shared MAC unit, the aggregate utilization of the shared MAC engine can be driven to 100%, i.e., full saturation (as reflected by both scenarios in the table).

3.2.3 Memory Subsystem.

A100's memory-system optimizations concentrate on two access paths: local memory and main memory. When viewed across different phases of LLM computation, the former primarily serves the compute-bound phases characterized by high arithmetic intensity and strong data reuse (i.e., the core vector/matrix operators), with the objective of keeping the working set resident in near-core storage as much as possible to provide stable data supply and minimize stalls. The latter primarily covers more memory-bound stages such as parameter reads and cache updates (especially the streaming access pattern during decode), with the goal of delivering robust sustained read bandwidth from the main-memory subsystem while providing sufficient latency-hiding capability.

The capacity provisioning of local memory is typically not the result of optimizing a single metric to its extreme; rather, it emerges from a multi-objective trade-off between performance benefit and implementation cost under the target system goals, where cost includes, but is not limited to, area, power, and bandwidth overhead. This configuration is also materially constrained by physical-implementation factors, such as frequency ceilings imposed by timing closure and area boundaries dictated by macro placement and interconnect routing. Accordingly, this design choice can be formalized as a canonical Pareto-optimal multi-objective optimization problem: selecting, within the feasible design space, a compromise

solution that is not simultaneously dominated by any other alternative across all objectives.

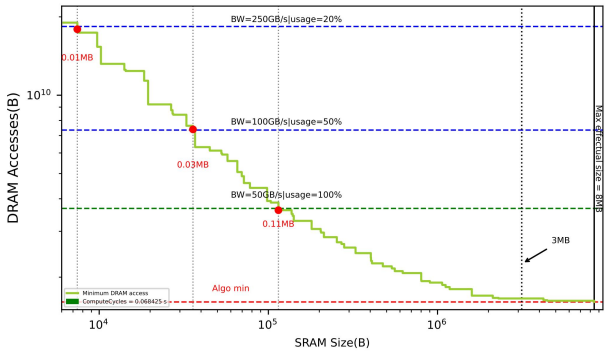

Figure 10: A100 local memory size analysis.

During the architectural design phase, we constructed a unified performance–cost model for multiple classes of representative workloads (including CNNs, LLMs, and ViTs), and performed parametric sweeps to identify the respective knee points, i.e., the operating region beyond which further increases in local-memory capacity yield sharply diminishing marginal performance returns while implementation cost continues to grow approximately linearly (or faster). Taking into account algorithmic evolution trends, target-application coverage, and implementation-cost constraints, we ultimately selected the knee point of Qwen-30B-A3B as the engineering reference for local-memory provisioning, thereby prioritizing—in terms of design objectives—the cache/working-set residency requirements of 30B–80B-scale models during inference.

The main-memory design is constrained in an engineering sense using Little's Law: in steady state, $lambda = \frac{WIP}{Lat}$.

Here, $lambda$ corresponds to sustainable throughput/bandwidth, WIP denotes the amount of in-flight concurrent requests, and Lat is the average service latency. The implication for main-memory optimization is direct: on the one hand, increase the maintainable WIP (i.e., more in-flight requests to cover DRAM latency and reduce bandwidth bubbles); on the other hand, reduce or amortize the effective Lat (i.e., mitigate conflicts and queuing delays).

A100's implementation follows these two principles. At the cluster level, the WIP scale is approximately 100, and the design adopts asymmetric sharing: a single core is not constrained by a fixed static quota and may borrow and occupy more than its equal-share portion of the in-flight window when the workload demands it. At the system level, further reductions in effective Lat are achieved via channel/link latency optimizations. Consequently, in memory-bound regimes, A100 is more likely to sustain high in-flight concurrency and to stably approach the achievable sustainable bandwidth.

3.3 Support for New RISC-V Features in K3

During the iterative evolution of our company's chip products, the previous-generation K1 chip integrated the X60 processor core, which fully complied with the ISA extension requirements defined by the RVA22 profile. In the new-generation K3 chip, the adopted X100 processor core further provides comprehensive support for all Mandatory Extensions required by the RVA23 specification. However, due to our distinctive scheduling architecture—under which AI workloads are dispatched exclusively to the A100 for computation, and the A100 core does not assume functions such as hypervisor management—the A100 compute core does not implement the RISC-V H extension (Hypervisor extension) in hardware. Nevertheless, aside from the H extension, all other mandatory extensions specified by the RVA23 standard are fully implemented in hardware on the A100.

We also made selective choices among the Optional Extensions in the RVA23 specification and implemented a subset of them, with the intent of ensuring strong forward-looking capability and adaptability across the chip's lifecycle.

3.3.1 Hypervisor.

Virtualization abstracts physical hardware resources such that multiple virtual machines (VMs) can execute concurrently on a single physical platform, thereby significantly improving resource utilization, strengthening isolation, and enabling flexible multi-tenant deployments. In the RISC-V ecosystem, virtualization is enabled via the Hypervisor extension (H-extension), which provides efficient hardware support for both CPU and memory virtualization. In addition, the Advanced Interrupt Architecture (AIA) introduces the IMSIC, enabling CPU cores to receive MSI interrupts and to deliver interrupts directly to guest VMs, substantially reducing interrupt-handling overhead and latency in virtualized environments.

On the K3 platform, the X100 core implements the Hypervisor extension, and each physical core can support up to seven active VMs. However, the two-stage address translation required by virtualization introduces additional translation overhead, which can materially impact application performance inside VMs. To mitigate this performance loss, the X100 core incorporates multiple optimizations at the translation lookaside buffer (TLB) level. First, the instruction TLB (ITLB) and data TLB (DTLB) directly cache translations from guest virtual addresses (GVA) to system physical addresses (SPA), such that, on an L1 TLB hit, virtualization incurs no additional translation latency. Second, the L2 TLB not only retains GVA→SPA mappings, but also caches guest physical address (GPA) to SPA mappings to expand address-translation coverage. Furthermore, the page-table-walker (PTW) caches store non-leaf page-table entries for GVA→GPA translation, accelerating page-table walks upon guest TLB misses. Collectively, these mechanisms aim to reduce memory-access latency and improve overall execution efficiency under virtualization. After enabling the above virtualization support, detailed performance comparisons for the X100 core are reported in Section 6.1 of this paper.

With respect to interrupt support, both the X100 core and the A100 core in the K3 platform support receiving external interrupts in the form of MSI. Specifically, on X100, both machine mode (M-mode) and supervisor mode (S-mode) support MSI external interrupt IDs 1–511; in addition, enabled by the Hypervisor extension, each VM running in virtual supervisor mode (VS-mode) can support direct MSI passthrough for interrupt IDs 1–63. For A100, M-mode and S-mode likewise support MSI external interrupt IDs 1–511; however, because this core does not implement the Hypervisor extension, it cannot provide VM-oriented interrupt passthrough.

3.3.2 *Vector Crypto.*

Cryptographic operations are fundamental for data security and communication privacy. However, scalar ISAs often face throughput and energy-efficiency bottlenecks for large-scale or high-throughput crypto workloads. To address this, RISC-V introduces Vector Cryptography extensions, providing vector-oriented crypto instructions that significantly improve parallelism and execution efficiency.

The extension covers both international-standard algorithms (e.g., AES, SHA-2) and Chinese commercial cryptography algorithms (e.g., SM4, SM3). By operating on structured vector element groups (Element Groups), it efficiently supports encryption/decryption, key expansion, and hashing. Its ISA design emphasizes data-independent execution latency, improving resistance to timing side-channel attacks. It also supports multiple configuration "suites" (e.g., Zvkn, Zvks), balancing flexibility with specialized hardware optimization.

In K3, both X100 and A100 implement Vector Crypto modules. They share the same implementation policy: SM3 and SHA2-64 use 256-bit element groups, while all remaining instructions use 128-bit element groups. With Vector Crypto enabled, typical encryption/decryption kernels can achieve up to 60× performance improvement over scalar execution; detailed data are reported in Section 6.1.

3.3.3 *Precision Conversion Extensions.*

To meet the growing demand for large-model training and inference, efficient low-precision compute has become critical. Both X100 and A100 are fully compatible with the RVA23 optional extension subsets for half-precision floating-point compute, including Zfh, Zvfh for FP16 scalar/vector operations, and Zfbfmin, Zvfbfmin, Zvfbfwma for BF16 and BF16 matrix operations. This unified ISA support ensures software compatibility and portability for foundational FP16/BF16 operations, providing essential low-precision compute capability for AI workloads.

Notably, A100 further integrates customized vector conversion instructions for FP8, supporting both E4M3 and E5M2 formats. FP8, as an emerging lower-precision format, can halve storage and transfer costs while maintaining acceptable numerical stability for key applications. Through tight microarchitectural integration, A100 implements these conversions into the execution pipeline, substantially reducing FP8 to FP16/BF16 conversion latency and improving conversion efficiency by orders of magnitude.

At the time of K3 design, RISC-V did not have an official OFP8 extension; as of this writing, RISC-V is defining a corresponding OFP8 standard. Once ratified, future cores will migrate to the official standardized extension.

Hardware-level precision conversion provides benefits beyond conversion speed. Modern AI models widely adopt mixed-precision strategies, since different layers/ops have different numerical sensitivity and may employ different quantization methods. A100's precision-conversion instructions enable dynamic, efficient on-demand precision switching across phases or compute units, greatly reducing the high overhead of software-emulated conversions. Because data can be stored at higher precision only when necessary—and otherwise converted at low cost—this effectively reduces off-chip bandwidth demand, which is crucial for bandwidth-bottlenecked AI acceleration scenarios.

4 Software Stack

4.1 Operating System

From a hardware-architecture perspective, although the X100 and A100 processors share a homogeneous core lineage, their functional roles differ substantially, forming a typical asymmetric compute architecture. X100 prioritizes general-purpose compute performance, whereas A100 is optimized specifically for AI and high-throughput vector computation. This specialization is reflected in both the ISA feature set and hardware configuration: X100 implements the Hypervisor extension and is suitable for general server scenarios requiring hardware-assisted virtualization; its vector length (VLEN) is 256 bits, targeting general-purpose vector processing. In contrast, A100 does not implement the Hypervisor extension, but increases VLEN to 1024 bits, enabling a significant throughput advantage for large-scale matrix operations and tensor computing.

Such asymmetry introduces challenges to OS-kernel scheduling. To exploit the flexibility of the fused heterogeneous platform while preserving deterministic execution behavior, we design and implement a core-affinity-based scheduling scheme. In the current system, all 16 cores across X100 and A100 are managed by the Linux kernel as an SMP system, and the OS exposes a unified 16-core logical view. However, the scheduling policy is explicitly partitioned: the 8 A100 cores are designated as AI-dedicated accelerator cores, serving only specific AI algorithms and vector-compute-intensive tasks, and are excluded from general-purpose workload handling. Accordingly, user threads are, by default, bound to the X100 core set at creation time. Even when the X100 set is saturated, the system does not migrate these workloads onto A100. Conversely, when a task (e.g., AI inference or algorithms optimized to leverage 1024-bit vectors) requires A100, the application must bind the corresponding threads to the A100 core set after thread creation. Once bound, these threads are constrained to execute only on A100 for their entire lifetime and are prohibited from migrating to X100.

While this policy addresses the baseline issues of mixed scheduling between X100 and A100—ensuring ISA compatibility (e.g., preventing a thread composed only of general instructions from being incorrectly scheduled onto an A100 core that supports only a subset of AI-related instructions, which could otherwise trigger exceptions)—it also introduces system- and engineering-level complexity. First, from a load-balancing perspective, the policy is not optimal: it may leave the A100 set idle while the X100 set remains fully utilized, reducing overall resource utilization. Second, cross-core migration is constrained by the need to strictly avoid scheduling mismatched vector contexts (e.g., different VLEN state) across core types, increasing scheduler complexity. Third, from a software-ecosystem standpoint, the current approach relies on modifying Linux kernel code, which is unfavorable for long-term maintenance and difficult to upstream into the mainline kernel community. In addition, application developers must understand the underlying hardware-aware scheduling behavior to optimize effectively, raising the development barrier.

To address these challenges, we are exploring an improved approach based on the Linux cpuset mechanism. This approach implements disciplined scheduling management by partitioning processor affinity domains: X100 cores (CPU0–7) are configured as the default domain, while A100 cores (CPU8–15) are configured as an

AI-exclusive domain. System workloads run in the default domain by default. Only when a task is explicitly labeled by the application as an AI task is it allowed to be placed into the AI-exclusive domain. This method shifts policy enforcement from kernel modifications to user-space configuration, leveraging Linux's native resource isolation and management framework, and is expected to provide clearer and more sustainable heterogeneous scheduling support while reducing overall system complexity.

4.2 AI Algorithms and Software Stack

An AI-DSA processor core that implements a complete set of RVV and tensor-extension ISAs runs on top of Linux. It exploits multi-thread scheduling to express the parallel, tile-based computation pattern in AI inference, while the general-purpose X100 cores regulate load balancing across A100 cores and provide fast-response synchronization, and can also handle selected asynchronous memory-transaction tasks. On this foundation, we build an AI compute software stack that integrates a hand-tuned operator library, an AI compiler, and a runtime system. The stack follows a pragmatic evolution path: first achieve peak performance for proprietary scenarios via hand-written kernels, then introduce compiler-based generation to generalize these performance wins. Concretely, the system is implemented as an inference engine based on ONNX Runtime with a private backend, together with a tile-based Triton operator compiler; both share the same runtime system as well as a unified threading and memory-optimization strategy.

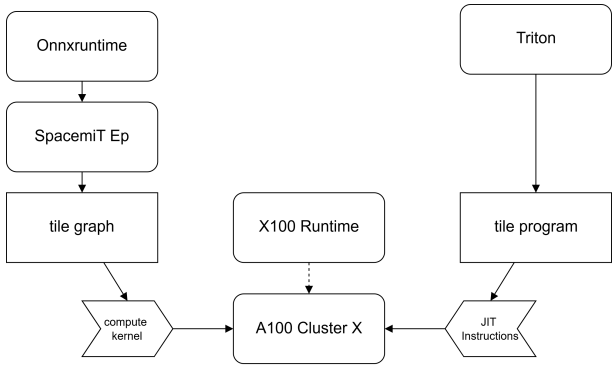

Figure 11: Software stack of the K3 AI compute system.

During the early phase of the K3 AI software-stack bring-up, we prioritize a hand-tuned operator layer together with a tile-level runtime scheduler, and interface it directly with the dominant inference ecosystem, ONNX Runtime. This enables rapid end-to-end enablement of a broad range of AI models, while accumulating reusable assets—including optimal instruction/compute sequences for hand-written kernels and multi-thread, multi-cluster scheduling policies—which in turn provide sufficient methodological grounding for a later transition to a more generalizable compiler-centric form of AI software.

4.2.1 *- End-to-end deployment form factor and engineering implementation highlights.*

Unified API operator library. At the outset of AI software development, we commit to two parallel tracks: a hand-written-kernel inference engine and an AI compiler. The operator-library APIs are aligned with MLIR meta-op definitions, which ensures semantic completeness and cross-framework compatibility. At the same time, it exposes a C API that early versions of the AI compiler can invoke. This allows the full compilation pipeline to be exercised before complete IR-based code generation is in place, which is a key enabler for K3 to bring up both AI inference and AI compilation rapidly in the early post-silicon phase.

X100 runtime system. Both the AI-compiler path and the inference-engine path rely on X100 to schedule AI workloads onto the two A100 clusters. In the inference abstraction, all operators—such as convolution and GEMM, as well as Triton-compiled kernels—use an A100 cluster as the hardware scheduling unit, which maps equivalently to a group of four A100 Linux threads. With two A100 clusters, the system forms two groups totaling eight A100 Linux threads, conceptually analogous to CUDA multi-stream execution. The X100 runtime dynamically dispatches tile-level compute tasks issued by the inference engine and by Triton onto the currently least-loaded A100-cluster thread group. In subsequent designs, K3 will further support distributed inference, where the runtime becomes topology-aware and schedules tile tasks in a locally optimal manner, thereby preserving a transparent programming model while improving the scalability of the inference engine.

5 Evaluation

5.1 CPU Performance

5.1.1 *Baseline Benchmarks.*

In the K3 design, general-purpose compute workloads are primarily executed by the X100 cores. To accurately evaluate and quantify the general-purpose processing capability of X100, we use the industry-standard SPEC CPU2006 integer suite (SPEC CINT2006) as the baseline benchmark. This suite consists of a set of integer, compute-intensive programs that approximate real application behaviors, and provides broad coverage of processor characteristics such as integer execution efficiency, branch prediction effectiveness, and memory-subsystem performance under diverse workloads including compilation, interpretation, file manipulation, and path searching.

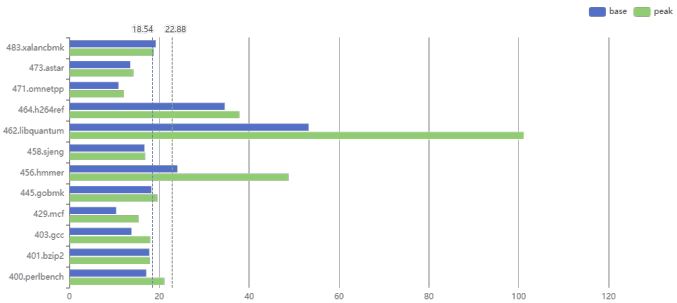

Figure 12: SPEC CPU 2006 of X100.

Figure 12 reports the overall single-core SPEC CINT2006 score of X100 as well as the per-benchmark score breakdown. For all

Table 5: SPEC2006 peak compare in VM.

benchmark	SPEC2006 peak	peak in VM	decrease(%)
400.perlbench	21.10	20.60	-2.33%
401.bzip2	17.83	17.72	-0.64%
403.gcc	17.95	16.15	-10.03%
429.mcf	15.38	12.84	-16.54%
445.gobmk	19.57	19.34	-1.16%
456.hmmer	48.72	48.65	-0.14%
458.sjeng	16.85	16.34	-3.03%
462.libquantum	101.11	98.48	-2.60%
464.h264ref	37.78	37.14	-1.69%
471.omnetpp	12.12	10.84	-10.53%
473.astar	14.20	12.85	-9.49%
483.xalancbmk	18.73	17.61	-5.99%
GEOMEAN	22.88	21.62	-5.48%

measurements, the X100 core runs at a fixed frequency of 2.4 GHz with the system memory configured as LPDDR5.

5.1.2 *Virtualization Performance.*

For virtualization performance evaluation, we use the same hardware configuration (X100 cores at 2.4 GHz with LPDDR5-6400) and quantify general-purpose compute performance under virtualization using the SPEC CPU2006 integer suite (CINT2006). The results show that the virtualization overhead varies significantly across benchmarks due to differences in computational characteristics, with performance losses ranging from 0.1% to 16%. Overall, virtualization reduces the aggregate SPEC CINT2006 score by 5.48%, as summarized in Table 5.

A closer examination of per-benchmark results indicates that the performance loss correlates strongly with the workload's memory-access behavior and control-flow complexity. Benchmarks with irregular memory-access patterns that limit the effectiveness of hardware prefetching (e.g., 429.mcf), as well as benchmarks with complex control flow and hard-to-predict branches (e.g., 403.gcc), exhibit more pronounced slowdowns under virtualization. The underlying cause is the increased penalty on TLB misses due to the additional address-translation stage introduced by virtualization (e.g., two-stage translation via mechanisms such as EPT or shadow page tables). When a workload's access stream cannot be effectively covered by the TLB, each translation miss triggers an extra mapping step from guest-physical to host-physical addresses, which increases effective memory-access latency and intensifies pressure on the memory subsystem, thereby further degrading performance.

5.1.3 *Cryptography Performance.*

To evaluate the performance impact of Vector Crypto on cryptographic workloads, we use OpenSSL as the benchmark suite, which integrates a broad range of cryptographic algorithm implementations and provides a standardized test environment. Specifically, we employ the built-in speed command in OpenSSL to benchmark multiple algorithms. This command quantifies throughput and operation latency across algorithms; during testing we sweep variable message sizes (e.g., from 16 bytes to 16,384 bytes) to emulate different workload regimes. To facilitate cross-algorithm comparison,

Table 6: OpenSSL Vector Crypto accelerate ratio.

benchmark	16 bytes	256 bytes	16384 bytes
chacha20	0.5	1.1	2.1
sha256	1.8	3.8	10.5
sha512	1.4	1.7	2.3
SM3	1.5	2.1	2.6
aes-128-cbc	4.7	12	13.9
aes-256-cbc	4.8	12.1	13.7
SM4	3.5	5.9	6.2
GHASH	2.5	21.3	46.6
hmac(sha256)	1.9	3.2	10.2

all results are normalized to the scalar baseline, i.e., the scalar implementation is set to 1, and optimized variants are reported as relative speedups.

The results clearly show that for algorithms such as AES, SHA, SM3, SM4, and GHASH, the Vector Crypto ISA defines dedicated acceleration instructions, yielding order-of-magnitude gains over both scalar implementations and generic vectorized implementations. Taking AES as an example, Vector Crypto introduces specialized table-lookup instructions that effectively eliminate cache-access overhead commonly associated with conventional lookup-table approaches; under large-message scenarios (e.g., 8 KB blocks), the normalized performance reaches nearly 14× of the scalar baseline. For GHASH, the X100/A100 architecture efficiently implements the Vector Crypto instruction vghsh. This instruction adopts a deeply pipelined design and completes each iteration in only 3 cycles, raising normalized performance to 46× of the scalar baseline, which highlights the advantage of Vector Crypto for specialized application domains.

In contrast, ChaCha20 is dominated by shifts and modular additions, which offers relatively limited vector-acceleration potential. It can only obtain modest gains from a small set of instructions such as vector rotate (vror). As a result, its advantage becomes pronounced only at sufficiently large message sizes (e.g., > 64 KB blocks), where throughput improvements from vector parallelism gradually amortize instruction-level overheads.

5.2 AI Performance

5.2.1 *Memory Read Bandwidth.*

For LLM inference, decode-phase TPS is often dominated by effective read bandwidth: generating each token triggers a full-model forward pass, and if weights/intermediate data cannot remain resident in cache over time, the bottleneck shifts from compute to the sustained throughput of reading data from memory.

A memory-access pattern that more closely matches the hardware objective of "fully utilizing the memory controllers" is thread-sharded multi-stream sequential read. In this pattern, a contiguous physical address range (or a page/block-partitioned region) is divided into N non-overlapping streams (chunks); N threads/cores are pinned, and each core linearly scans its own stream with cacheline strides (optionally assisted by prefetching). The streams run in parallel with minimal barrier synchronization, continuously issuing read requests so that the memory controllers can saturate the read

Table 7: A100 Main Memory Bandwidth.

	Read Bandwidth	Relative to normal
normal	10139	1.00x
vector	32412	3.20x
opt v1	32654	3.22x
opt v2	33226	3.28x
opt v3	32360	3.19x
opt all	33214	3.28x

datapath via queuing and reordering. This pattern is prefetcher-friendly, increases MLP (memory-level parallelism), and scales well with core count, typically approaching the peak bandwidth of an LPDDR5 interface. In the LLM decode phase, A100's parameter reads are, in essence, organized as per-layer weight accesses structured into multi-core, segmented linear streams, using this multi-stream sequential-read pattern to continuously drive the memory system toward bandwidth saturation.

Below we report the measured bandwidth under the K3 test setup by progressively enabling A100's read-bandwidth-related hardware optimization knobs (system-level optimizations are still in progress; the results below do not represent final peak performance):

5.2.2 Advantages of 1024-bit Vectors.

With RVV configured at VLEN=1024 and 32 architectural vector registers, the programming model can be viewed as providing an effective 4 KB ultra-fast local scratchpad, which enables stronger locality optimizations in certain specialized scenarios.

Norm-style broadcast computation. This pattern typically takes the form C = A[..., C] * Scale[C], where the channel dimension C is often 128/256/512/768. With VLEN=1024, 16 vector registers are sufficient to hold the Scale vector for C < 512, so the Scale tensor only needs to be loaded once globally. This reduces pressure on the L1/L2 caches by avoiding repeated scale fetches during the main compute loop.

FlashAttention. In FlashAttention, the hidden dimension is commonly 128 or 64. Under FP16 precision, two vector registers can hold the hidden dimension for one sequence chunk. Even without tensor-extension instructions, RVV alone can eliminate the inner loop over the hidden dimension, and keep the $V \cdot QK$ accumulation resident in 2-4 vector registers, making the compute path largely independent of L1/L2 caching during the accumulation phase.

Reducing the likelihood of VPR spills. During compiler register allocation, code that heavily uses LMUL=4 or LMUL=8—especially in transcendental-function approximations such as exp and tanh—is prone to vector register spills, which degrades the instruction schedule and overall performance. With VLEN=1024, using LMUL=1 can provide a compute width equivalent to VLEN=256 with LMUL=4, substantially reducing the probability of vector-register spills and improving kernel robustness.

5.2.3 Typical CNN Performance.

A 1024-bit vector datapath enables higher per-core data-level parallelism and improves the efficiency of tile-level parallel execution for compute-dense kernels (e.g., convolution/GEMM). In practice, this translates into higher single-stream throughput and stronger scaling when increasing software parallelism (threads) and modestly increasing batch size, as shown Table 8.

Table 8: Typical CNN performance.

Model/Inference Config	Batch = 1 Thread = 1	Batch = 1 Thread = 4	Batch = 2 Thread = 4
ResNet50	52	129	165
RepVGG	207	451	570
YOLOv8s	9	31	44

6 Acknowledgements

We thank the RISC-V open-source ecosystem, which enabled the K3 chip to rapidly integrate into a relatively complete and mature software stack. We also thank the organizations and companies contributing to the open ecosystem. The baseline X100 implementation is based on Xuantie's open-source OpenC910; the baseline RT24 real-time core is based on openhwgroup's open-source CVA6. During X100 convergence and iteration, we utilized the Xiangshan toolchain, including the xs-gem5 modeling tool for front-end modeling, architecture exploration, and performance analysis; Xiangshan's NEMU-based slicing flow, DRAMSim environment, and top-down analysis tools to accelerate benchmark analysis and iteration. We also leveraged a large set of community open-source tools and test suites, including but not limited to spike, force, rv-dv, rv-arch-tests, to accelerate convergence of K3 general-purpose CPU IP, AI CPU IP, and real-time CPU IP.

References

TBD